

MCA
(SEM II) THEORY EXAMINATION 2022-23
DATABASE MANAGEMENT SYSTEMS

Time: 3 Hours

Total Marks: 100

Note: Attempt all Sections. If require any missing data; then choose suitably.

SECTION A

1. Attempt all questions in brief.

2 x 10 = 20

(a) Define the terms Data and Database.

☐ **Data:** Data is defined as raw, unorganized facts that need to be processed to make them meaningful. Data is represented as characters, numbers, or symbols that are stored and transmitted by a computer. It has no inherent meaning until it is interpreted (e.g., "101" is data; "Room 101" is information).

☐ **Database:** A database is a structured, organized collection of interrelated data stored in a way that it can be easily accessed, managed, and updated. A database represents some aspect of the real world and is designed for a specific purpose, such as a Banking Database or a University Database.

(b) Discuss various advantages of DBMS?

A Database Management System (DBMS) offers several improvements over traditional file systems:

- ☐ **Reduced Data Redundancy:** Unlike file systems, DBMS ensures that data is not duplicated unnecessarily, saving storage and preventing inconsistencies.
- ☐ **Data Sharing:** Multiple users and applications can access the same data simultaneously under controlled conditions.
- ☐ **Data Integrity and Security:** DBMS provides constraints (like Data Types or Primary Keys) to ensure data accuracy and uses schemas to restrict unauthorized access.
- ☐ **Backup and Recovery:** It provides automatic subsystems to recover data in case of hardware or software failures.
- ☐ **Data Independence:** It separates the metadata (structure) from the actual data, meaning changes in the database schema do not require changes in the application programs.

(c) Define cursor in PL/SQL.

A Cursor is a pointer to a private memory area (Context Area) allocated by the Oracle server. It is used to handle the result set of a SELECT statement that returns multiple rows, allowing the program to process each row individually. It acts as a pointer to the result set of a query. It is of two types.

- **Implicit Cursors:** Automatically generated by Oracle for all DML statements (INSERT, UPDATE, DELETE) and single-row SELECT statements.
 - **Explicit Cursors:** Defined by the programmer for queries that returns multiple rows. The lifecycle involves four steps:
 1. **DECLARE:** Naming the cursor and defining the SQL.
 2. **OPEN:** Allocating memory and executing the query.
 3. **FETCH:** Retrieving rows one by one.
 4. **CLOSE:** Releasing the memory.
-

(d) What do you mean by sub-queries?

A **Sub-query** (or Nested Query) is a query within another SQL query. It is typically embedded in the WHERE, HAVING, or FROM clause of a main SELECT, INSERT, UPDATE, or DELETE statement to filter data based on the results of the inner query.

(e) Why normalization is used?

Normalization is a multi-step process of organizing data in a database to achieve two main goals:

1. **Eliminate Redundant Data:** Storing the same data in multiple places wastes space and complicates updates.
 2. **Ensure Data Dependency:** Data should be logically stored so that it is only dependent on the primary key. By applying Normal Forms (1NF, 2NF, 3NF, BCNF), we prevent **Anomalies**:
 - **Insertion Anomaly:** Inability to add data because some other data is missing.
 - **Deletion Anomaly:** Unintended loss of data when a record is deleted.
 - **Update Anomaly:** Inconsistency caused by updating only one instance of duplicated data.
-

(f) What is pseudo-transitivity?

Pseudo-transitivity is one of **Armstrong's Axioms** used in database design to derive new functional dependencies from existing ones. Pseudo-transitivity is an inference rule which states:
If $X \rightarrow Y$ and $WY \rightarrow Z$, then $WX \rightarrow Z$.

(g) Define serializable schedule.

A schedule is a sequence of operations (read, write, commit, abort) from various transactions. A **Serializable Schedule** is a non-serial schedule that produces the same result as some serial execution of the same transactions.

- It is the highest level of **Isolation**.
 - It ensures that even though transactions are interleaved for performance, the database remains in a consistent state, as if they were running one after the other.
 - There are two types: **Conflict Serializability** and **View Serializability**.
-

(h) What do you mean by transaction failure?

A transaction failure occurs when a transaction is unable to complete its execution and reach the "Committed" state. It is classified into:

- **Logical Errors:** The transaction cannot continue due to internal conditions like bad input, data not found, or overflow.
- **System Errors:** The DBMS terminates the transaction because of an external condition like a **Deadlock** (where two transactions wait for each other) or lack of system resources (CPU/RAM). When a failure occurs, the system must perform a **Rollback** to maintain the **Atomicity** of the transaction.

(i) Define concurrent processing in database.

Concurrent processing refers to the ability of a DBMS to handle multiple transactions simultaneously. In a multi-user system, operations from different transactions are interleaved.

- **Purpose:** To increase **Throughput** (number of transactions per second) and decrease **Response Time**.
 - **Challenge:** It requires **Concurrency Control** (using techniques like Locking or Timestamps) to prevent problems like the Lost Update problem or Dirty Reads.
-

(j) What do you mean by dispatcher in terms of Oracle?

In an Oracle **Shared Server Architecture**, a **Dispatcher** is a crucial background process that manages communication between client processes and server processes.

1. Instead of a dedicated server process for every user, the Dispatcher receives requests from multiple clients.
 2. It places these requests into a **Common Request Queue** in the System Global Area (SGA).
 3. Available shared server processes pick up requests from this queue.
 4. Once processed, the Dispatcher picks the result from a **Response Queue** and delivers it back to the specific client. This significantly reduces memory overhead.
-

SECTION B

2. Attempt any *three* of the following:

10 x 3 = 30

(a) Discuss ANSI/SPARC architecture in detail.

The ANSI/SPARC architecture (American National Standards Institute, Standards Planning and Requirements Committee) is a fundamental framework for database design. It proposes a **three-level architecture** to separate the user applications from the physical database, primarily to achieve **Data Independence**.

The Three Levels:

1. **External Level (View Level):**
 - This is the highest level, closest to the users.
 - It describes only that part of the database that is relevant to a specific user group.
 - Multiple external views can exist for the same database (e.g., an HR view vs. an Accounting view).
2. **Conceptual Level (Logical Level):**
 - This level describes what data is stored in the database and the relationships among that data.
 - It includes the logical structure (tables, constraints, business rules) without describing physical storage.
 - It is maintained by the DBA (Database Administrator).
3. **Internal Level (Physical Level):**
 - The lowest level, describing how data is actually stored on storage disks.
 - It deals with data compression, encryption, hashing, and indexing techniques.

Data Independence:

- **Logical Data Independence:** The ability to change the conceptual schema without changing external views or application programs.
- **Physical Data Independence:** The ability to change the physical/internal schema (e.g., moving from HDD to SSD) without affecting the conceptual schema.

(b) Create a table Employee(Emp_id, Emp_name, Emp_add, Emp_basicpay) and insert some records in it. Create another table Emp_detail(Emp_name, Emp_add) from Employee table by using SELECT statement.

Create table:

```
SQL> CREATE TABLE Employee (Emp_id NUMBER(4) PRIMARY KEY, Emp_name  
VARCHAR(50), Emp_add VARCHAR(20), Emp_basicpay NUMBER(10, 2));
```

Insert Records:

```
SQL> INSERT INTO Employee VALUES (101, 'Rahul Sharma', 'New Delhi', 55000);  
SQL> INSERT INTO Employee VALUES (102, 'Priya Verma', 'Mumbai', 62000);  
SQL> INSERT INTO Employee VALUES (103, 'Amit Goel', 'Lucknow', 48000);
```

3. Create Emp_detail from Employee using SELECT: This is often called a Computed Table or a "Create Table As Select" (CTAS) operation.

```
SQL> CREATE TABLE Emp_detail AS SELECT Emp_name, Emp_add FROM Employee;
```

(c) Explain basic inference rules applied on the set of functional dependencies. Suppose R with attributes A, B, C, D, E, F have the FDs $A \rightarrow BC$, $B \rightarrow E$, $CD \rightarrow EF$. Prove that FD : $AD \rightarrow F$ also holds in R.

Inference Rules (Armstrong's Axioms) and Proof

Inference rules are used to derive new Functional Dependencies (FDs) from a given set.

Basic Inference Rules:

1. **Reflexivity:** If $Y \subseteq X$, then $X \rightarrow Y$.
2. **Augmentation:** If $X \rightarrow Y$, then $XZ \rightarrow YZ$ for any Z.
3. **Transitivity:** If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.
4. **Decomposition:** If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$.
5. **Union:** If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$.

Proof:

Given: $F = \{ A \rightarrow BC, B \rightarrow E, CD \rightarrow EF \}$

To Prove: $AD \rightarrow F$ holds.

Proof Steps:

1. From $A \rightarrow BC$ (Given), by **Decomposition**, we get $A \rightarrow B$ and $A \rightarrow C$.
2. Apply **Augmentation** on $A \rightarrow C$ with D: Add D to both sides, so $AD \rightarrow CD$.
3. We are given $CD \rightarrow EF$.
4. By **Transitivity** between $AD \rightarrow CD$ and $CD \rightarrow EF$, we get **$AD \rightarrow EF$** .

5. By **Decomposition** on $AD \rightarrow EF$, we get **$AD \rightarrow E$ and $AD \rightarrow F$** .
(Hence Proved)

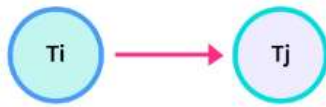
(d) Write short note on Testing of serializability.

Testing for serializability is crucial to ensure that concurrent transactions do not leave the database in an inconsistent state. The most common method is the **Precedence Graph (Conflict Graph)**.

Steps to Test Conflict Serializability:

1. **Identify Nodes:** Create a node for every transaction ($T_1, T_2, \dots, T_n$) in the schedule.
2. **Draw Edges:** Draw a directed edge $T_i \rightarrow T_j$ if the two transactions have a "conflict":
 - They access the same data item Q .
 - At least one of them is a $Write(Q)$ operation.
 - T_i executes its operation before T_j .
3. **Analyze the Graph:**
 - **If the graph is Acyclic (no cycles):** The schedule is **Serializable**.
 - **If the graph contains a Cycle:** The schedule is **Not Serializable**, meaning it may lead to data inconsistency.

Precedence Graph for Schedule S



(e) What are various types of Locks applied on data items? Discuss Two-phase locking protocol and Strict Two-phase locking protocols.

Locking Protocols in DBMS: Locking is a mechanism to control concurrent access to data.

Types of Locks:

- **Shared Lock (S):** Also called a Read-lock. Multiple transactions can hold a shared lock on the same item. If T_1 has an S-lock, T_2 can also get an S-lock, but cannot get an X-lock.
- **Exclusive Lock (X):** Also called a Write-lock. Only one transaction can hold an X-lock on an item. If T_1 has an X-lock, no other transaction can get any lock (S or X) on that item.

Two-Phase Locking (2PL) Protocol:

This protocol requires that each transaction issue lock and unlock requests in two phases:

1. **Growing Phase:** Transaction may obtain locks but may not release any.
 2. **Shrinking Phase:** Transaction may release locks but may not obtain any new ones.
- **Drawback:** 2PL ensures serializability but can lead to **Deadlocks** and **Cascading Rollbacks**.

Strict Two-Phase Locking (Strict 2PL):

Strict 2PL adds a specific rule to the basic 2PL:

- **Rule:** A transaction must hold all its **Exclusive (X) locks** until it commits or aborts.
- **Advantage:** It prevents **Cascading Rollbacks**. If T1 modifies data, no other transaction can read that data until T1 is finished, ensuring that if T1 fails, no other transaction is "poisoned" by its uncommitted data.

SECTION C

3. Attempt any *one* part of the following:

10 x 1 = 10

(a) Discuss various types of Entity, Attribute and Relation used in designing an ER diagram.

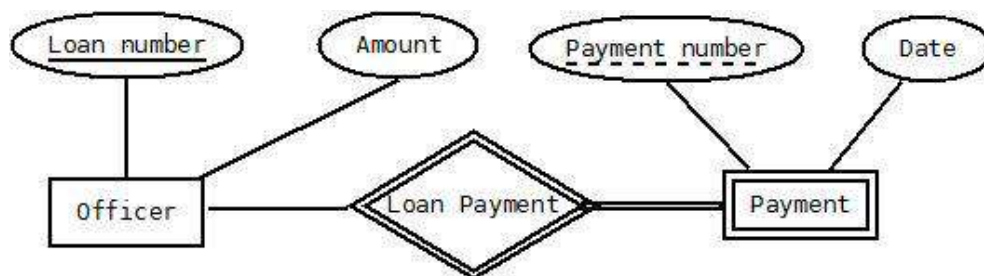
In Database Management Systems, the Entity-Relationship (ER) diagram serves as a blueprint for the database.

1 Entities

An Entity is an object or concept in the real world that is distinguishable from all other objects (e.g., a specific student, a car, or a department).

- **Strong Entity:** An entity that has a primary key and does not depend on any other entity for its existence. It is represented by a **single rectangle**.
 - *Example:* Student (with Roll_No as a primary key).
- **Weak Entity:** An entity that does not have a primary key of its own and depends on a "strong" entity for its identification. It is represented by a **double rectangle**.
 - *Example:* Dependent of an employee. If the employee leaves the company, the dependent record has no meaning.

In the below ER Diagram, Payment is the weak entity. Loan Payment is the identifying relationship and Payment Number is the partial key. Primary Key of the Loan along with the partial key would be used to identify the records.

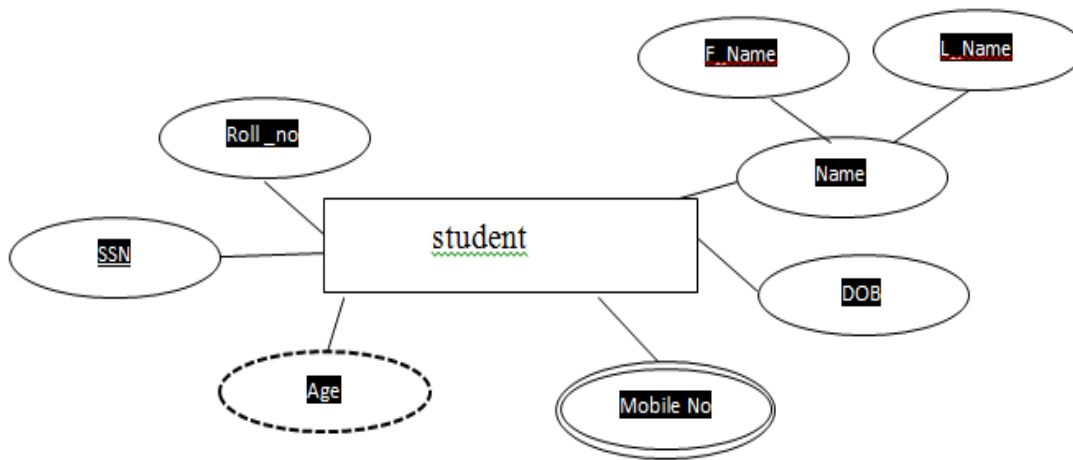


2. Attributes

Attributes are the properties or characteristics that describe an entity. They are represented by **ellipses** in an ER diagram.

- **Simple Attribute:** An attribute that cannot be further subdivided into smaller components.
 - *Example:* Roll_Number or Salary.
- **Composite Attribute:** An attribute that can be divided into smaller sub-parts, which represent more basic attributes.
 - *Example:* Name can be divided into First_Name and Last_Name.
- **Single-Valued Attribute:** An attribute that holds only one value for a specific entity instance.
 - *Example:* Age or Date_of_Birth.
- **Multi-Valued Attribute:** An attribute that can have more than one value for a single entity. It is represented by a **double ellipse**.
 - *Example:* Phone_Number (a person can have multiple numbers).
- **Derived Attribute:** An attribute whose value is calculated from other related attributes. It is represented by a **dashed ellipse**.

- *Example:* Age derived from Date_of_Birth.
- **Key Attribute:** An attribute that uniquely identifies an entity in an entity set. It is represented by an ellipse with the text **underlined**.
 - *Example:* Social_Security_Number.



3. Relationships

A **Relationship** defines how two or more entities are associated with each other. It is represented by a **diamond** symbol.

Relationship Types by Degree:

- **Unary Relationship:** A relationship where the same entity participates twice (recursive).
 - *Example:* An Employee manages another Employee.
- **Binary Relationship:** A relationship between two distinct entities. This is the most common type.
 - *Example:* Student enrolls in Course.
- **Ternary Relationship:** A relationship involving three different entities.

Relationship Types by Cardinality:

Cardinality defines the number of instances of one entity that can be associated with instances of another entity.

One-to-One (1:1): One instance of Entity A is associated with exactly one instance of Entity B. Person has one Passport.

One-to-Many (1:N) : One instance of Entity A can be associated with many instances of Entity B. Department has many Employees.

Many-to-Many: Many instances of Entity A can be associated with many instances of Entity B. Students register for many Courses.

(b) What are various types of databases? Discuss them.

1. Relational Databases (RDBMS)

Relational databases are the most traditional and widely used type. Data is organized into structured **tables** consisting of rows and columns.

- **Key Feature:** They use **SQL** (Structured Query Language) and focus on the **ACID** properties (Atomicity, Consistency, Isolation, Durability) to ensure data integrity.
- **Data Model:** Based on the relational model where tables are linked using primary and foreign keys.
- **Examples:** MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server.

2. Non-Relational Databases (NoSQL)

NoSQL databases are designed to handle **unstructured or semi-structured data**, high velocity, and massive scaling. Unlike RDBMS, they do not require a fixed schema.

- **Document Databases:** Store data in JSON-like documents (e.g., MongoDB, CouchDB).
- **Key-Value Stores:** Data is stored as a simple pair of a unique key and an associated value (e.g., Redis, DynamoDB).
- **Column-Family Stores:** Optimized for queries over large datasets, storing data in columns rather than rows (e.g., Apache Cassandra, HBase).
- **Graph Databases:** Focus on the relationships between data points using nodes and edges (e.g., Neo4j).

3. Distributed Databases

In a distributed database, data is not stored in a single physical location. Instead, it is spread across multiple computers or sites, which may be geographically dispersed.

- **Homogeneous:** All sites use identical software and hardware.
- **Heterogeneous:** Different sites may use different hardware and operating systems.
- **Benefit:** Provides high availability and fault tolerance; if one site fails, others can continue to function.

4. Centralized Databases

This is the opposite of a distributed database. All data is stored in a **single location** (e.g., a mainframe or a central server).

- **Benefit:** Easier to manage, update, and back up because everything is in one place.
- **Drawback:** If the central system fails, all users lose access (Single Point of Failure).

5. Cloud Databases

These are databases built and accessed through a cloud platform (e.g., AWS, Azure, or Google Cloud).

- **Deployment:** Can be "IaaS" (where you manage the DB on a virtual machine) or "DBaaS" (Database as a Service, where the provider handles all maintenance).
- **Benefit:** Elasticity (scaling up or down instantly) and pay-per-use pricing.

6. Object-Oriented Databases (OODBMS)

These databases store data in the form of **objects**, similar to how data is represented in object-oriented programming (OOP) languages like Java or C++.

- **Usage:** Used when the data is very complex (e.g., 3D modeling, engineering designs, or multimedia) where mapping objects to tables would be too slow.

7. Hierarchical and Network Databases (Legacy)

- **Hierarchical:** Data is organized in a tree-like structure where each child node has only one parent (e.g., IBM's IMS).

- **Network:** An evolution of the hierarchical model where a child node can have multiple parents, forming a graph-like structure.
-

4. Attempt any one part of the following: 10 x 1 = 10

a) Create the tables Supplier(SNo, SName, City), Parts(PNo, PName, Color, City), and Shipment(SNo, PNo, Quantity). Answer the following queries :

```
CREATE TABLE Supplier (  
    SNo VARCHAR(10) PRIMARY KEY,  
    SName VARCHAR(50),  
    City VARCHAR(50)  
);  
  
CREATE TABLE Parts (  
    PNo VARCHAR(10) PRIMARY KEY,  
    PName VARCHAR(50),  
    Color VARCHAR(20),  
    City VARCHAR(50)  
);  
  
CREATE TABLE Shipment (  
    SNo VARCHAR(10),  
    PNo VARCHAR(10),  
    Quantity INT,  
    FOREIGN KEY (SNo) REFERENCES Supplier(SNo),  
    FOREIGN KEY (PNo) REFERENCES Parts(PNo)  
);
```

(i) Get the total number of suppliers.

SQL> SELECT COUNT(SNo) AS TotalSuppliers FROM Supplier;

(ii) Get the total number of suppliers currently supplying parts.

SQL> SELECT COUNT(DISTINCT (SNo)) AS ActiveSuppliers FROM Shipment;

(iii) Get part numbers of all parts supplied.

SQL> SELECT DISTINCT PNo FROM Shipment;

(iv) Change the color of parts from red to black.

SQL> UPDATE Parts SET Color = 'Black' WHERE Color = 'Red';

(b) Discuss Primary Key, Unique Key, Candidate Key and Foreign Key by taking examples.

1. Candidate Key

A **Candidate Key** is any column (or set of columns) that contains unique information and could potentially serve as the primary identifier for a row. A table can have multiple Candidate Keys.

- **Example:** In an `Employees` table, both **Employee_ID** and **Social_Security_Number** are unique for every person. Both are "candidates" for the job of being the main identifier.

2. Primary Key

The **Primary Key** is the "chosen one." Out of all the Candidate Keys, you pick one to be the main identifier for the table.

- **The Rules:** It must be unique, and it **cannot be NULL**.
- **Example:** From our candidate list above, we select **Employee_ID** as the Primary Key because it's easier to manage than a sensitive SSN.

3. Unique Key

A **Unique Key** ensures that all values in a column are different. It is very similar to a Primary Key, but with one major difference: it **allows for one NULL value** (depending on the database system).

- **The Use Case:** We use this for data that *should* be unique but isn't the main way to link tables.
- **Example:** In a `Users` table, the `Email_Address` should be a Unique Key. we don't want two people with the same email, but the email isn't necessarily the internal ID used to track the user.

4. Foreign Key

A **Foreign Key** is a column that points to the Primary Key of *another* table. Its job is to create a link between the two tables and maintain **referential integrity**.

- **Example:** Imagine a `Departments` table and an `Employees` table.
 - In the `Departments` table, `Dept_ID` is the Primary Key.
 - In the `Employees` table, we add a column called `Dept_ID`.
 - This `Dept_ID` in the `Employees` table is a **Foreign Key**. It ensures that you can't assign an employee to a department that doesn't exist.
-

5. Attempt any *one* part of the following: $10 \times 1 = 10$

(a) Given a relation $R(P, Q, R, S, T)$ and Functional Dependency set $FD = \{ PQ \rightarrow R, S \rightarrow T \}$, determine whether the given R is in 2NF? If not convert it into 2 NF.

To determine if the relation $R(P, Q, R, S, T)$ is in **Second Normal Form (2NF)**, we first need to identify the **Candidate Key** and then check for **Partial Functional Dependencies**.

Step 1: Find the Candidate Key

A Candidate Key must be able to functionally determine all other attributes in the relation.

1. **Check attributes on the Left Hand Side (LHS):** $\{P, Q, S\}$ are on the LHS of the FDs.
2. **Check attributes missing from Right Hand Side (RHS):** $\{P, Q, S\}$ never appear on the RHS, meaning they **must** be part of any Candidate Key.
3. **Calculate the Closure of $\{P, Q, S\}$:**
 - $\{P, Q, S\} = \{P, Q, S\}$
 - From $PQ \rightarrow R$, we add $R: \{P, Q, S, R\}$
 - From $S \rightarrow T$, we add $T: \{P, Q, S, R, T\}$

Since the closure contains all attributes, the **Candidate Key is $\{P, Q, S\}$** .

- **Prime Attributes:** P, Q, S (Parts of the candidate key)
- **Non-Prime Attributes:** R, T

Step 2: Check for 2NF Violation

A relation is in **2NF** if every non-prime attribute is **fully** functionally dependent on the primary key. If a non-prime attribute depends on only a *part* of the candidate key, it is a **Partial Dependency**, which violates 2NF.

1. **$PQ \rightarrow R$:** R is non-prime. PQ is a proper subset of the candidate key $\{P, Q, S\}$. This is a **Partial Dependency**.
2. **$S \rightarrow T$:** T is non-prime. S is a proper subset of the candidate key $\{P, Q, S\}$. This is a **Partial Dependency**.

Conclusion: The relation R is **not in 2NF**.

Step 3: Convert to 2NF (Decomposition)

To convert to 2NF, we decompose the relation into smaller tables to eliminate partial dependencies. Each partial dependency will form its own table.

Table 1: $R1(P, Q, R)$

We take the dependency $PQ \rightarrow R$

- **Attributes:** $\{P, Q, R\}$
- **Primary Key:** $\{P, Q\}$

Table 2: R2(S, T)

We take the dependency $S \rightarrow T$.

- **Attributes:** {S, T}
- **Primary Key:** S

Table 3: R3(P, Q, S)

We must keep a table that relates the parts of the original Primary Key so that no data/relationship is lost.

- **Attributes:** {P, Q, S}
- **Primary Key:** {P, Q, S}

This decomposition ensures that every non-prime attribute is now fully dependent on the primary key of its respective table.

(b) How will you check that the decomposition of a relation on a given set of functional dependencies is dependency preserving? Discuss with the help of an example.

A decomposition is **dependency preserving** if the union of the functional dependencies (FDs) that hold on the individual decomposed relations is equivalent to the set of FDs in the original relation.

If we break a big table into smaller ones, we shouldn't "lose" any of the original rules. We should still be able to enforce all original constraints without having to join the tables back together.

The Test for Dependency Preservation

To check if a decomposition of R into R1, R2, ..., Rn is dependency preserving for a set of functional dependencies F:

1. For each FD $X \rightarrow Y$ in the original set F, check if it is "covered" by the decomposition.
2. An FD $X \rightarrow Y$ is covered if it is present in the union of closures of the decomposed relations: $(F1 \cup F2 \dots \cup Fn)$.
3. **The Practical Method:** For each $X \rightarrow Y$ in F:
 - Set result = X.
 - Repeat until result doesn't change:
 - For each decomposed relation Ri:
 - $result = result \cup ((result \cap Ri)^+ \cap Ri)$
 - If the final result contains Y, the dependency $X \rightarrow Y$ is preserved.
 - 4. If **all** original FDs are preserved, the decomposition is dependency preserving.

Example

Let's take a relation R(A, B, C, D) with the following FDs:

- $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D\}$

Suppose we decompose R into:

- R1(A, B)
- R2(B, C)
- R3(C, D)

Step 1: Identify local FDs for each relation

- R1 contains $A \rightarrow B$
- R2 contains $B \rightarrow C$
- R3 contains $C \rightarrow D$

Step 2: Check the original FDs against the union

The union of local FDs is $F' = \{A \rightarrow B, B \rightarrow C, C \rightarrow D\}$.

Since F' is exactly the same as F, **all dependencies are preserved.**

6. Attempt any one part of the following:

10 x 1 = 10

(a) Explain ACID properties of a transaction in detail.

In database management systems, a **Transaction** is a single logical unit of work. To ensure the integrity of the data—especially during crashes or concurrent updates—every transaction must follow the **ACID** properties.

1. Atomicity ("All or Nothing")

Atomicity treats a transaction as a single, indivisible unit. Either the entire transaction succeeds, or none of it is applied to the database. There is no "middle ground."

- **Mechanism:** If any part of the transaction fails (due to a power outage, error, or constraint violation), the database performs a **rollback**, undoing every change made during that specific transaction.
- **Example:** In a bank transfer from Account A to Account B, two things must happen: debiting A and crediting B. If the system crashes after debiting A but before crediting B, Atomicity ensures Account A gets its money back.

2. Consistency

Consistency ensures that a transaction takes the database from one valid state to another valid state, maintaining all predefined rules, including constraints, cascades, and triggers.

- **Mechanism:** The database validates the data against "Integrity Constraints." If a transaction would result in a state that violates a rule (like a negative balance in a field that requires positive integers), the transaction is aborted.
- **Example:** If a database rule states that the total "Sum of all accounts" must remain constant during a transfer, Consistency ensures that the transfer doesn't accidentally "create" or "destroy" money.

3. Isolation

Isolation ensures that concurrent transactions (multiple people using the database at once) do not interfere with each other. Each transaction should feel as though it is the only one running on the system.

- **Mechanism:** This is usually handled via **Concurrency Control** (like Locking or Timestamping). It prevents "Dirty Reads" (reading uncommitted data from another user).
- **Example:** If two people try to book the last seat on a flight at the exact same millisecond, Isolation ensures that the system processes them sequentially. One person gets the seat, and the other sees it as "Sold Out" once their transaction starts.

4. Durability

Durability guarantees that once a transaction has been committed, it will remain committed even in the event of a system failure (e.g., a power outage or disk crash).

- **Mechanism:** Once the user receives a "Success" message, the changes are recorded in non-volatile memory (like a hard drive) and usually written to a **Transaction Log**. Even if the server restarts immediately after, the database can use these logs to restore the data.
 - **Example:** Once your bank app says "Transfer Successful," you don't have to worry about the money disappearing if the bank's servers lose power five seconds later
-

(b) Write short note on Log-Based Recovery.

When a database crashes, the system uses **Log-Based Recovery** to restore the data to a consistent state. The "Log" is a sequence of records stored on stable storage (disk) that tracks every single operation performed by transactions.

The golden rule is the **Write-Ahead Logging (WAL)** protocol: no data is updated on the physical disk until the corresponding log record has been written to stable storage.

Types of Log Records

Every log entry typically contains:

- **Transaction ID:** Which transaction performed the update.
- **Data-Item ID:** Which record was changed.
- **Old Value:** The value before the update (used for **Undo**).
- **New Value:** The value after the update (used for **Redo**).

Recovery Techniques

There are two primary strategies for log-based recovery depending on when the database actually writes changes to the disk:

A. Deferred Database Modification: In this approach, a transaction does not update the physical database until it partially commits. All changes are recorded in the log first.

If a crash occurs: The system only needs to **Redo** transactions that reached the "Commit" state. Since the actual database wasn't touched before the commit, no "Undo" is necessary.

B. Immediate Database Modification: The database can be updated while the transaction is still active (before it commits).

If a crash occurs:

- **Undo:** Any transaction that was active but not committed must be rolled back using the "Old Values."
- **Redo:** Any transaction that was committed must be reapplied using the "New Values."

The Role of Checkpoints

If a database has been running for months, the log file becomes massive. In a crash, it would take hours to re-process the entire log. **Checkpoints** solve this.

Periodically, the system performs a checkpoint:

1. It stops all transactions temporarily.
2. It flushes all log records and modified data from volatile RAM to the physical disk.
3. It writes a <checkpoint> record to the log.

During recovery, the system only needs to look at the log records appearing *after* the last successful checkpoint. Everything before that is guaranteed to be safely on the disk.

The Recovery Process: When the system restarts after a crash, it scans the log:

1. **Redo List:** Contains transactions that have both a <start> and a <commit> (or <abort>) record. The system reapplies these to ensure Durability.
2. **Undo List:** Contains transactions that have a <start> record but **no** <commit> record. The system reverses these to ensure Atomicity.

7. Attempt any one part of the following:

10 x 1 = 10

(a) What is Timestamp Ordering Protocol? Discuss working of Basic Timestamp ordering protocol along with advantages and disadvantages of it.

The **Timestamp Ordering Protocol** is an optimistic concurrency control mechanism used to determine the serializability order of transactions. Unlike locking, which makes transactions "wait in line," timestamping assigns a unique, fixed priority to each transaction based on when it entered the system.

The Core Concept: Each transaction T_i is assigned a unique **Timestamp**, denoted as $TS(T_i)$.

- If T_i starts before T_j , then $TS(T_i) < TS(T_j)$.
- For every data item Q , the system maintains two values:
 - **W-timestamp(Q):** The largest timestamp of any transaction that successfully executed $write(Q)$.
 - **R-timestamp(Q):** The largest timestamp of any transaction that successfully executed $read(Q)$.

Working of Basic Timestamp Ordering

The protocol ensures that if $TS(T_i) < TS(T_j)$, the database acts as if T_i finished before T_j started. If a transaction tries to perform an operation that violates this "chronological order," the operation is rejected and the transaction is rolled back.

A. When T_i issues $read(Q)$:

1. **Conflict Check:** If $TS(T_i) < W\text{-timestamp}(Q)$, it means a "younger" transaction has already overwritten the value T_i was supposed to read.
 - o **Action:** Reject $read(Q)$ and **Rollback** T_i .
2. **Success:** If $TS(T_i) \geq W\{timestamp\}(Q)$:
 - o **Action:** Execute $read(Q)$ and update $R\text{-timestamp}(Q) = \max(R\text{-timestamp}(Q), TS(T_i))$.

B. When T_i issues $write(Q)$:

1. **Read Conflict:** If $TS(T_i) < R\text{-timestamp}(Q)$, it means a "younger" transaction has already read the old value, and T_i 's write would invalidate that read.
 - o **Action:** Reject $write(Q)$ and **Rollback** T_i .
2. **Write Conflict:** If $TS(T_i) < W\text{-timestamp}(Q)$, it means T_i is trying to write an obsolete value.
 - o **Action:** Reject $write(Q)$ and **Rollback** T_i .
3. **Success:** Otherwise:
 - o **Action:** Execute $write(Q)$ and set $W\text{-timestamp}(Q) = TS(T_i)$.

Advantages

- **Deadlock Free:** Since transactions only wait for their "turn" based on time and never wait for each other in a circular fashion, deadlocks are impossible.
- **Ensures Serializability:** It guarantees that the schedule will be equivalent to a serial schedule ordered by timestamps.
- **High Concurrency:** It works well in systems where conflicts are rare (optimistic approach), as it doesn't require the overhead of maintaining locks.

4. Disadvantages

- **Starvation:** If a long transaction is repeatedly rolled back because younger, faster transactions keep conflicting with it, it may never complete (Starvation).
- **Cascading Rollbacks:** If T_i is rolled back, any transaction that read data written by T_i must also be rolled back, leading to a "domino effect."
- **Storage Overhead:** The system must store two timestamps for every single data item in the database, which can be memory-intensive.

(b) Write short note on Multiple Granularity.

In a database system, **Multiple Granularity** is a locking strategy that allows data items of various sizes to be locked. Instead of locking only individual records or locking the entire database (which stops everyone else from working), Multiple Granularity provides a hierarchical approach to concurrency control.

The Hierarchy of Data: Think of a database as a tree-like structure. Multiple Granularity allows a transaction to lock a node at any level:

1. **Database** (Highest level)
2. **Areas/Files**
3. **Tables/Relations**
4. **Pages/Blocks**
5. **Records/Tuples** (Lowest level)

Intention Locks: The biggest challenge with hierarchical locking knows if a "child" node is already locked by someone else. If I want to lock an entire Table, I shouldn't have to check every single Record to see if someone else is using it.

For this we use Intention Locks. Before a transaction locks a node, it must place an intention lock on all the "ancestor" nodes (parents, grandparents).

- **Intention-Shared (IS):** Indicates that a transaction intends to lock individual descendants in Shared (S) mode.
- **Intention-Exclusive (IX):** Indicates that a transaction intends to lock descendants in Exclusive (X) mode.
- **Shared and Intention-Exclusive (SIX):** The entire subtree is locked in S mode, but explicit X locking will be done on some descendants.

How it Works

To lock a node, a transaction must follow these steps:

1. **Root First:** The transaction must first lock the root of the tree.
2. **Parent Requirement:** To get an **S** or **IS** lock on a node, the parent must be locked in at least **IS** mode.
3. **Parent Requirement for X:** To get an **X**, **IX**, or **SIX** lock on a node, the parent must be locked in at least **IX** mode.
4. **Bottom-Up Release:** Locks are released from the leaf nodes upward to the root.

Multiple Granularity balances **Overhead:**

- **Fine Granularity (Locking a Record):**
 - *Pro:* High concurrency (many people can edit the same table).
 - *Con:* High overhead (the system has to manage thousands of tiny locks).
- **Coarse Granularity (Locking a Table):**
 - *Pro:* Low overhead (only one lock to manage).
 - *Con:* Low concurrency (no one else can touch any record in that table).

If a transaction needs to generate a report on all 1,000,000 employees, it's more efficient for the system to grant one **S lock** on the **Employee Table**. If a transaction only needs to change one employee's phone number, it's better to grant an **X lock** on that specific **Record** while placing an **IX lock** on the Table level.
